

CDS 230 · TOPIC 17 · TEACHING LESSON

Computing with Randomness

Stochastic Models: Monte Carlo & Markov Chains



Faculty Candidate · Department of Computational & Data Sciences · George Mason University



LET'S START WITH A QUESTION

How Long Does It Take to Drive From New York to Florida?

Take a guess — shout out a number.

18 hours

20 hours

24 hours

But wait...

What about traffic? Rain? Construction? Rest stops? Every trip is different — so is there really just one answer?

NEW YORK

~1,200 miles

FLORIDA



LESSON PLAN

What You'll Be Able to Do in 20 Minutes

1

Explain the idea

State the Monte Carlo principle: estimate a quantity by sampling and counting.

2

Write the code

Implement the π estimator in Python with a loop, then vectorize it with NumPy.

3

Connect the theory

Tie results to the Law of Large Numbers, prediction error, and full distributions.

4

See the whole family

Extend from independent sampling (Monte Carlo) to step-by-step Markov chains.



THE METHOD

What Is Monte Carlo?

When a problem is too tangled to solve with a formula, Monte Carlo swaps the math for an **experiment**: generate random inputs, run them through a model, and repeat — thousands of times. The statistics of those runs become your answer.

- Works when nothing else does

High dimensions, messy systems, no clean equation — sampling still works.

- Gives you the whole picture

A single estimate (like π) or an entire distribution (like our road trip).



Why “Monte Carlo”?

Physicists at Los Alamos invented the method in the 1940s to model how neutrons scatter — problems with no tidy solution.

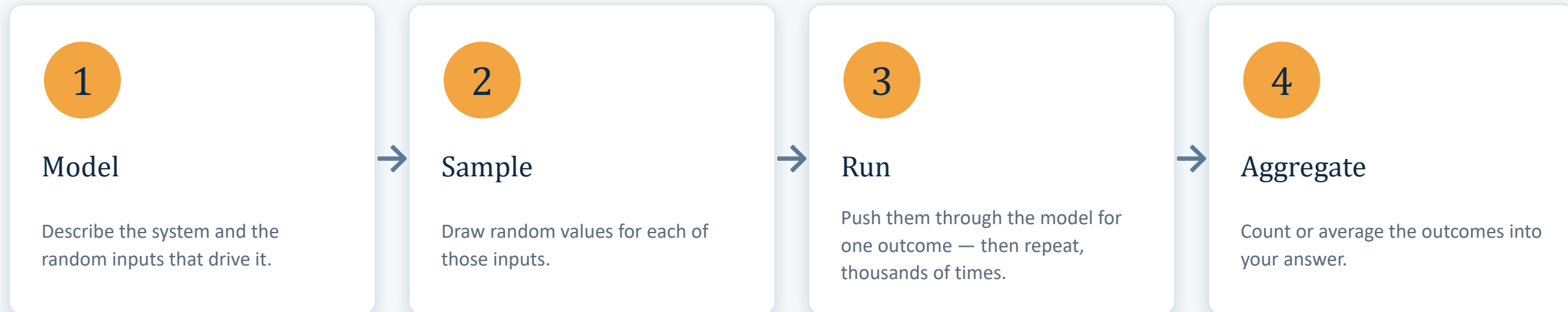
They named it after the famous casino — because the whole idea runs on chance.



ONE RECIPE, EVERY PROBLEM

Every Monte Carlo Simulation Is the Same Four Steps

Learn this pattern once and you can simulate almost anything — coin flips, stock prices, epidemics, or a drive to Florida.



The answer can be a single number (π) or a whole distribution (**travel time**) — *same four steps either way. Let's watch them work.*



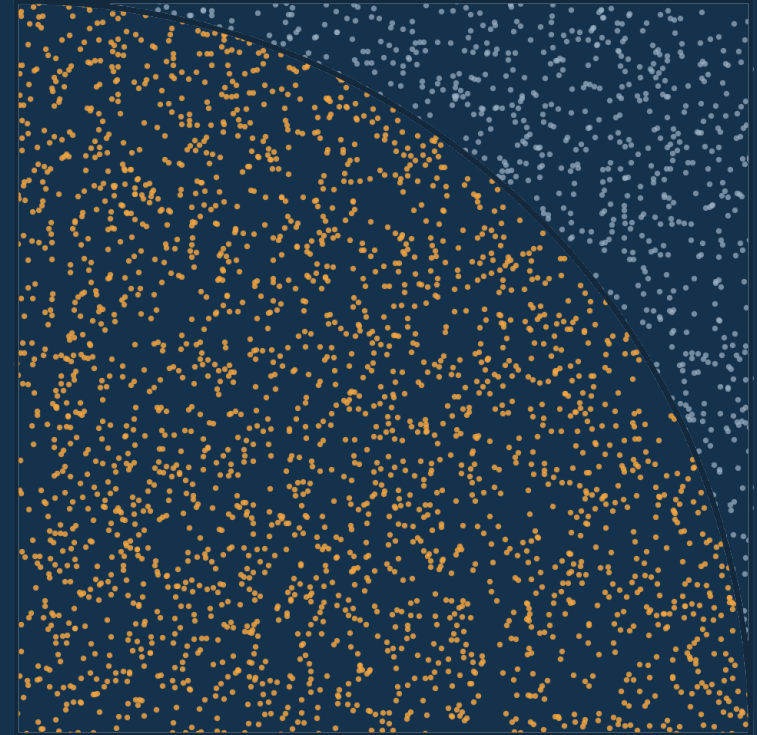
APPLICATION EXAMPLE · ESTIMATING π

Run the Recipe on a Number We Already Know

Let's start with π — we already know the answer — so we can check whether the method works.

Darts thrown at a square with a quarter-circle on it.

- 1 Most darts land somewhere in the square.
- 2 Some land inside the curved region; some land outside it.
- 3 The fraction that lands inside is the curve's share of the square's area.
- 4 Area is something we can turn into π — so counting darts gives us π .



3,000 random darts



THE MATHEMATICS

Why Counting Darts Gives You π

SET THE STAGE

Use the unit square, $0 \leq x, y \leq 1$, with a quarter circle of radius 1.

TWO AREAS

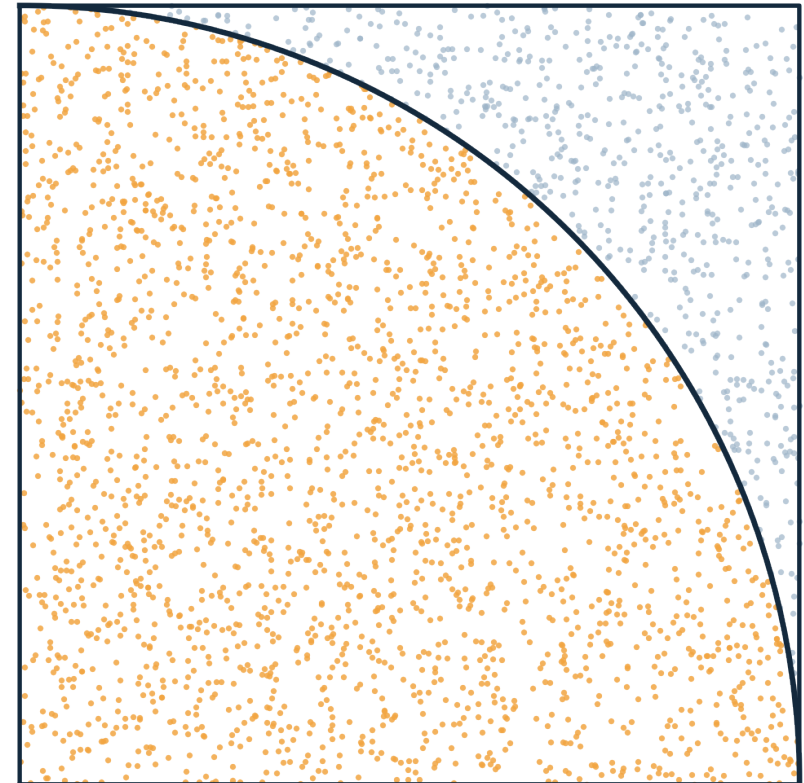
Square area = $1 \times 1 = 1$. Quarter-circle area = $\pi r^2/4 = \pi/4$.

THE RATIO

A random dart lands inside with probability $(\pi/4) \div 1 = \pi/4$.

So if a fraction p of darts land inside:

$$\pi \approx 4 \times p = 4 \times (\text{darts inside} / \text{total darts})$$



● inside = count these ● outside



LIVE CODING · STEP 1

First, the Loop



```
import random

def estimate_pi(n):
    inside = 0
    for _ in range(n):
        x = random.random()      # point in [0,1)
        y = random.random()
        if x*x + y*y <= 1.0:     # inside?
            inside += 1
    return 4 * inside / n
```

Talking through it

`random.random()`

draws a number uniformly in $[0, 1)$ — our source of randomness.

The if test

$x^2 + y^2 \leq 1$ is exactly “inside the circle.” The geometry from the last slide, in one line.

The loop

does the experiment N times and keeps a running tally — nothing fancy.

`estimate_pi(1_000_000)` → **3.1416**



LIVE CODING · STEP 2

Now Make It Fast with NumPy



```
import numpy as np

def estimate_pi(n):
    x = np.random.random(n)      # n points
    y = np.random.random(n)      # at once
    hits = np.sum(x**2 + y**2 <= 1.0)
    return 4 * hits / n
```



Same logic, no Python loop. NumPy runs the comparison on all N points in compiled C — often 50–100× faster, and it reads like the math.

Loop vs. vectorized

Readability

Loop spells out the experiment; NumPy mirrors the formula $4 \cdot \text{hits} / n$ directly.

Speed

Loop: one point per iteration. NumPy: all points in a single vectorized operation.

Teaching arc

Students see the idea first, then learn why we vectorize — the core NumPy lesson.

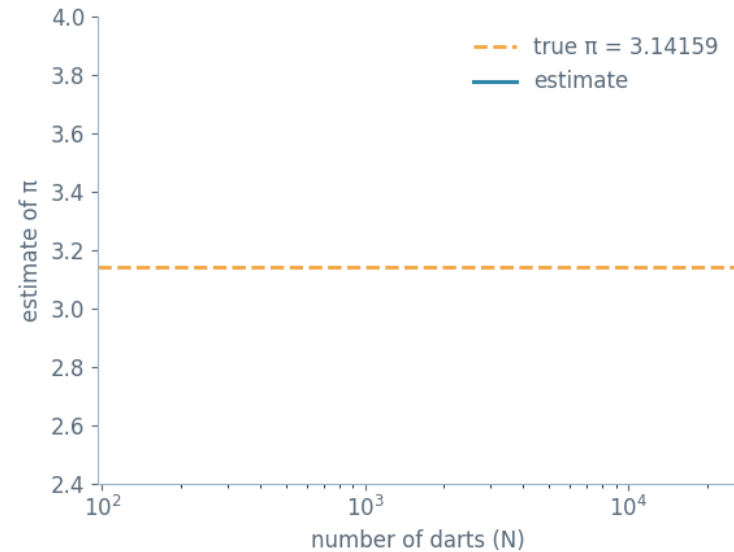
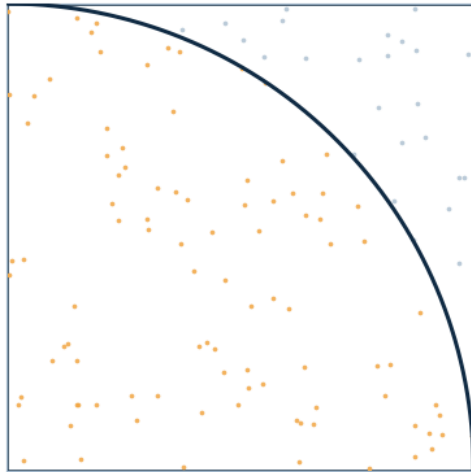


WHAT HAPPENS WHEN WE RUN IT

More Darts, Better Answer

Monte Carlo: estimating π by throwing random darts

darts: 120 estimate: 3.1667



The Law of Large Numbers

At $N = 10$ the estimate is wild (it bounced to 4.0).

By $N = 1,000,000$ it settles to

3.14177

vs. $\pi = 3.14159$

Averaging more independent trials drives the random noise down toward the true value.



THINKING CRITICALLY

How Fast Does the Error Shrink?

Monte Carlo error shrinks like $1/\sqrt{N}$. To get one more decimal place of accuracy, you need about **100× more darts**.

~100

darts

error around 0.06

~10k

darts

error around 0.02

~1M

darts

error around 0.0002

So why use it?

That $1/\sqrt{N}$ rate doesn't care how many dimensions you're in — so in high-dimensional problems, Monte Carlo wins where grids fail.

Error $\approx 0.8 / \sqrt{N}$ — diminishing returns, but dimension-proof



CALLBACK · REMEMBER OUR QUESTION?

Now Let's Actually Simulate That Drive

Instead of guessing one number, we sample thousands of possible trips — each with its own random conditions.
One run = one possible trip; the same darts-and-counting idea, applied to a messy real system.

THE RANDOM INGREDIENTS

● Cruising speed

Normal around 62 mph — driver, road, and weather all nudge it.

● Traffic

Random congestion: usually ~1 h, occasionally a lot more.

● Rain

35% chance; adds 0.5–2 h when it hits.

● Construction

50% chance; adds 0.2–1.5 h of crawling.

● Rest stops

A break every ~3 h, 15–35 min each.



```
N = 50_000; dist = 1200      # miles, NY->FL
speed  = rng.normal(62, 4, N) # avg mph
drive  = dist / speed         # hours
traffic = rng.exponential(1.0, N) # jams
rain    = where(p<.35, U(.5,2), 0)
build   = where(p<.50, U(.2,1.5), 0)
rest    = (drive//3) * U(.25,.55) # stops

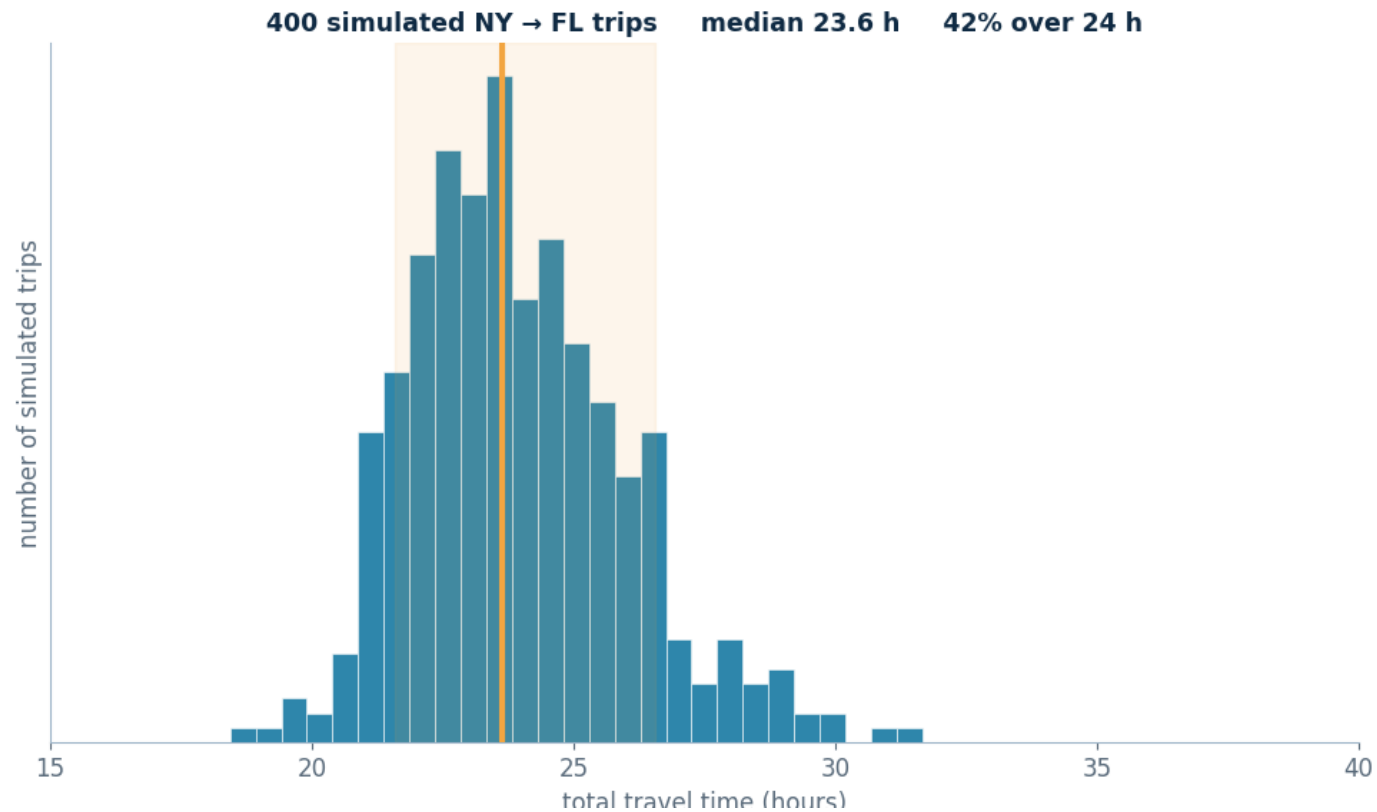
total = drive + traffic + rain + build + rest
```

Vectorized — all 50,000 trips at once, exactly like the NumPy π estimator.



THE PAYOFF

There's No Single Answer — There's a Distribution



50,000 simulated New York → Florida trips

What the simulation says

23.6

hours — the median trip

21.2–26.3 h

where 8 in 10 trips land (P10–P90)

41%

of trips take more than 24 hours

18–38 h

full range the model ever produced

The honest answer isn't a number — it's this shape. And that long right tail is exactly why it's really a two-day drive.



YOUR TURN · ACTIVE LEARNING

Five Minutes, Three Challenges

1

Predict

Before running it: if I use only $N = 4$ darts, what estimates of π are even possible?
(Hint: list the values $4 \cdot k/4$ can take.)

2

Modify

Change one line so the program also reports the error $|\text{estimate} - \pi|$. Which line, and where?

3

Extend

Our traffic chain assumed the next state depends only on the current one. Name one situation where that “memoryless” assumption breaks down.

Talk to your neighbor — we'll take answers on #1 and live-code #2 together.



Adding Memory: Markov Chains

Our road-trip model flipped an **independent** coin for rain on every trip. But weather has memory — a rainy day makes another rainy day more likely than chance. A **Markov chain** captures that: a system that hops between states, where the next one depends only on the current one.

States

A few conditions the system can be in — here: Sunny, Cloudy, Rainy.

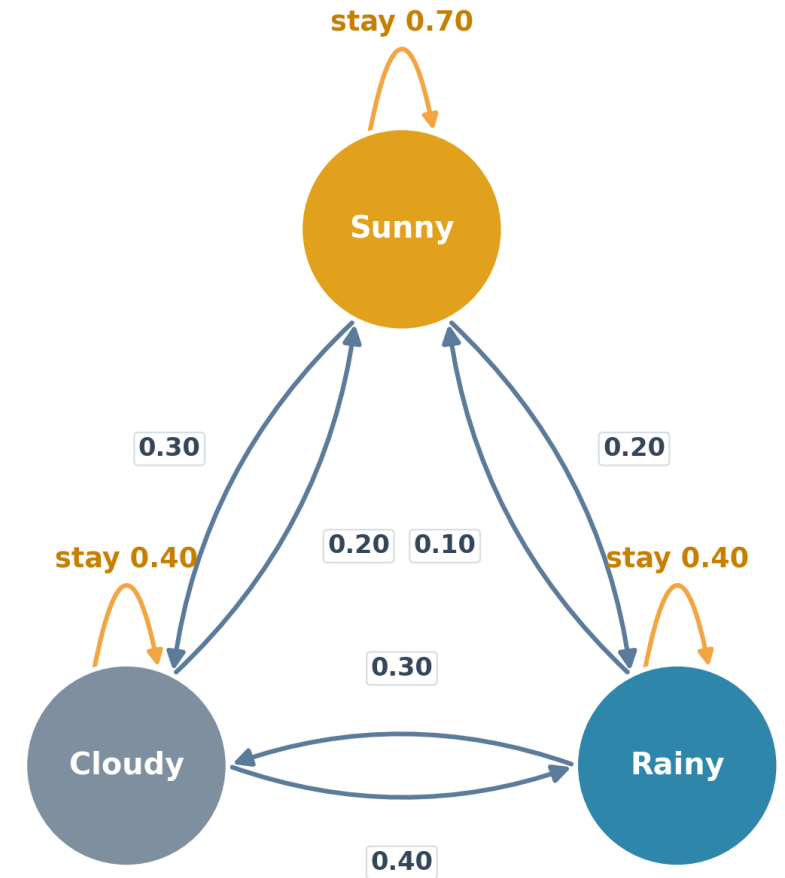
Transitions

Each day, you move to the next state with a fixed probability.

Memoryless

Only today matters — not the whole week behind you.

The same darts-and-counting engine — now the random draw is “what’s the weather tomorrow?”



Day-to-day weather — arrows show transition odds



FROM ONE STEP TO THE LONG RUN

Where Does It Settle? Why These Two Belong Together



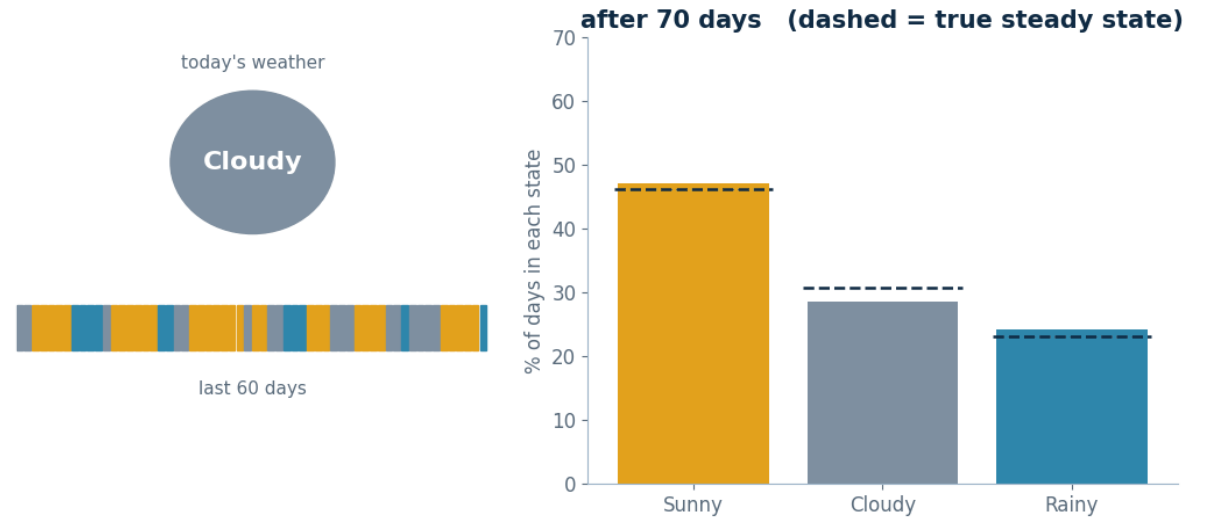
```
state = "Rainy"; counts = {}  
for _ in range(20_000): # days  
    counts[state] += 1  
    state = rng.choice(# tomorrow  
                      next_states, p=P[state])
```

Step the chain with Monte Carlo — each hop is one random draw.

Another convergence story

Start sunny or start rainy — it doesn't matter. Run the chain long enough and the fraction of days in each state always settles to the same **steady state** (about 46% sunny, 31% cloudy, 23% rainy). It's the Markov-chain cousin of the Law of Large Numbers.

Markov chain: weather that remembers yesterday



Two tools, one family

Monte Carlo = independent samples to estimate. **Markov chains** = dependent steps over time.

Combine them → Markov Chain Monte Carlo (MCMC), the engine behind modern Bayesian inference and AI. That's why the syllabus files both under *Stochastic Models*.



WORKED EXAMPLE · 6 DAYS

Six Days, One Random Roll at a Time

Each day, take **yesterday's** bands (right), roll a random u , and read off which band it lands in. We start on a rainy day.

DAY	WEATHER	ROLL u	RESULT
1	Rainy	—	Starting day
2	Rainy	0.805	Yesterday Rainy → Rainy
3	Rainy	0.808	Yesterday Rainy → Rainy
4	Cloudy	0.515	Yesterday Rainy → Cloudy
5	Sunny	0.286	Yesterday Cloudy → Sunny
6	Sunny	0.054	Yesterday Sunny → Sunny

THE BANDS · ONE 0–1 LINE PER STATE

Yesterday's weather picks the line; the roll picks the segment.

● if yesterday = Sunny



● if yesterday = Cloudy



● if yesterday = Rainy



Day 4 example: yesterday was Rainy, so use the Rainy line; the roll 0.515 lands in the Cloudy segment [0.20–0.60) → Cloudy. Each roll looks only at the day before — that's the Markov rule.



Three Challenges — Worked Answers

1

With $N = 4$ darts, which π estimates are possible?

Only **0, 1, 2, 3, 4** — the estimate equals **k** (darts inside), because $4 \cdot k / 4 = k$.

→ Best you can reach is 3 (error ≈ 0.14) — you can never land on 3.14. That coarse spacing is exactly why we need thousands of darts.

2

One line so it also reports the error $|\text{estimate} - \pi|$

Right after `pi_est = 4*inside/N`, add: **`error = abs(pi_est - np.pi)`**

```
import numpy as np
def estimate_pi(n):
    x = np.random.random(n)      # n points
    y = np.random.random(n)      # at once
    hits = np.sum(x**2 + y**2 <= 1.0)
    pi_est = 4 * hits / n
    error = abs(pi_est - np.pi)
    return error
```

→ It has to go after the estimate (you need the value first), and it works only because we know π — the luxury that makes π a great teaching example.



LOOKING AHEAD

My Teaching Goals in the CDS Department

I want every CDS undergraduate to leave with two things — the applied skills to land a job, and enough of a taste of research to choose that path if it calls them.



Code-first foundations, from day one

Students build and run real programs in every class — the way today's lesson did — not just watch them.



GOAL 1

Industry-ready: solving real problems

- Courses built around real-world examples.
- Hands-on, with real data and tools.
- I bring genuine problems from DoD-funded AI and health informatics straight into the classroom.

→ feeds internships (CDS 491), the capstone (CDS 492), and the region's employers



GOAL 2

Research-curious: a taste of discovery

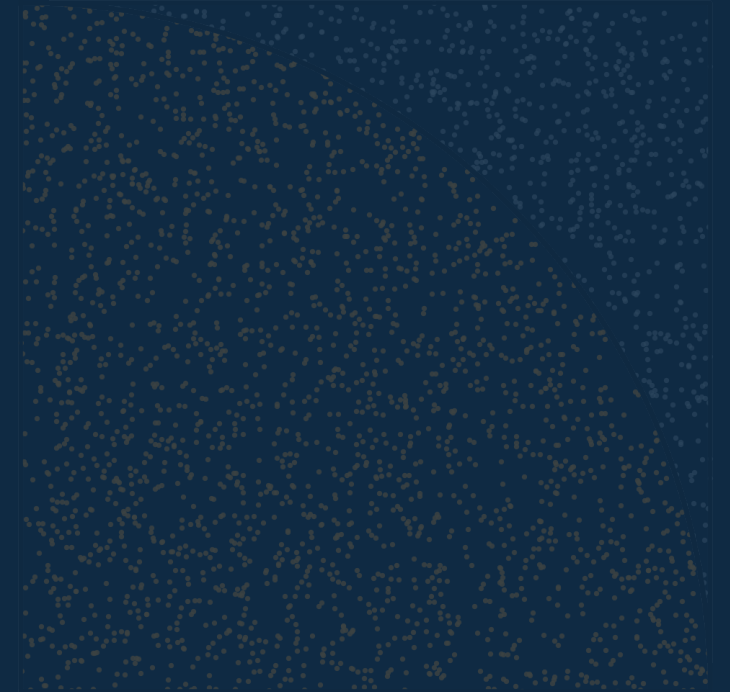
- Small research components in courses
- Mentored undergraduate summer projects — with the honest aim of helping interested students reach their first paper, so those drawn to research can find the path.

→ feeds directed research (CDS 490) and the BS → MS → PhD pipeline

Thank You

Questions, pushback, and “what would you do in week 10?” all welcome.

Teaching Materials : https://github.com/srbnghosh99/Faculty_Candidate_Lecture



Stochastic Models: Monte Carlo & Markov Chains · CDS 230 · George Mason University



ZOOMING OUT

One Lesson, the Whole Toolkit

This single example exercises almost every thread of the course — which is exactly why it makes a great anchor problem.

Random numbers

Generators and uniform sampling are the engine of the whole method.

Loops & NumPy

From an explicit loop to vectorized arrays — the central programming arc.

Probability

Areas become probabilities; the Law of Large Numbers makes it converge.

Stochastic models

Both halves of topic 17 — Monte Carlo and Markov chains — meeting as MCMC.

Visualization

The dartboard and convergence plot turn abstract numbers into intuition.

Scientific judgment

Estimating error and cost is how computational scientists trust a result.